```
singleton = class << "cat"; self; end
singleton.new
```

*produces:*

```
        from prog.rb:2:in `<main>'
prog.rb:2:in `new': can't create instance of singleton class (TypeError)
```

Let's tie together what we know about instance variables, self, and singleton classes. We could write class-level accessor methods to let us get and set the value of an instance variable defined in a class object. Ruby already has attr_accessor, which defines getter and setter methods. Normally, though, these are defined as instance methods and hence will access values stored in instances of a class. To make them work with class-level instance variables, we have to invoke attr_accessor in the singleton class:

```
class Test
  @var = 99

  class << self
    attr_accessor :var
  end
end

puts "Original value = #{Test.var}"
Test.var = "cat"
puts "New value = #{Test.var}"
```

*produces:*

```
Original value = 99
New value = cat
```

## Inheritance and Visibility

There's a wrinkle to when it comes to method definition and class inheritance, but it's fairly obscure. Within a class definition, you can change the visibility of a method in an ancestor class. For example, you can do something like this:

```
class Base
  def a_method
    puts "Got here"
  end
  private :a_method
end

class MakeItPublic < Base
  public :a_method
end

class KeepItPrivate < Base
end
```

In this example, you would be able to invoke a_method in instances of class MakeItPublic but not via instances of Base or KeepItPrivate.

So, how does Ruby pull off this feat of having one method with two different visibilities? Simply put, it cheats.